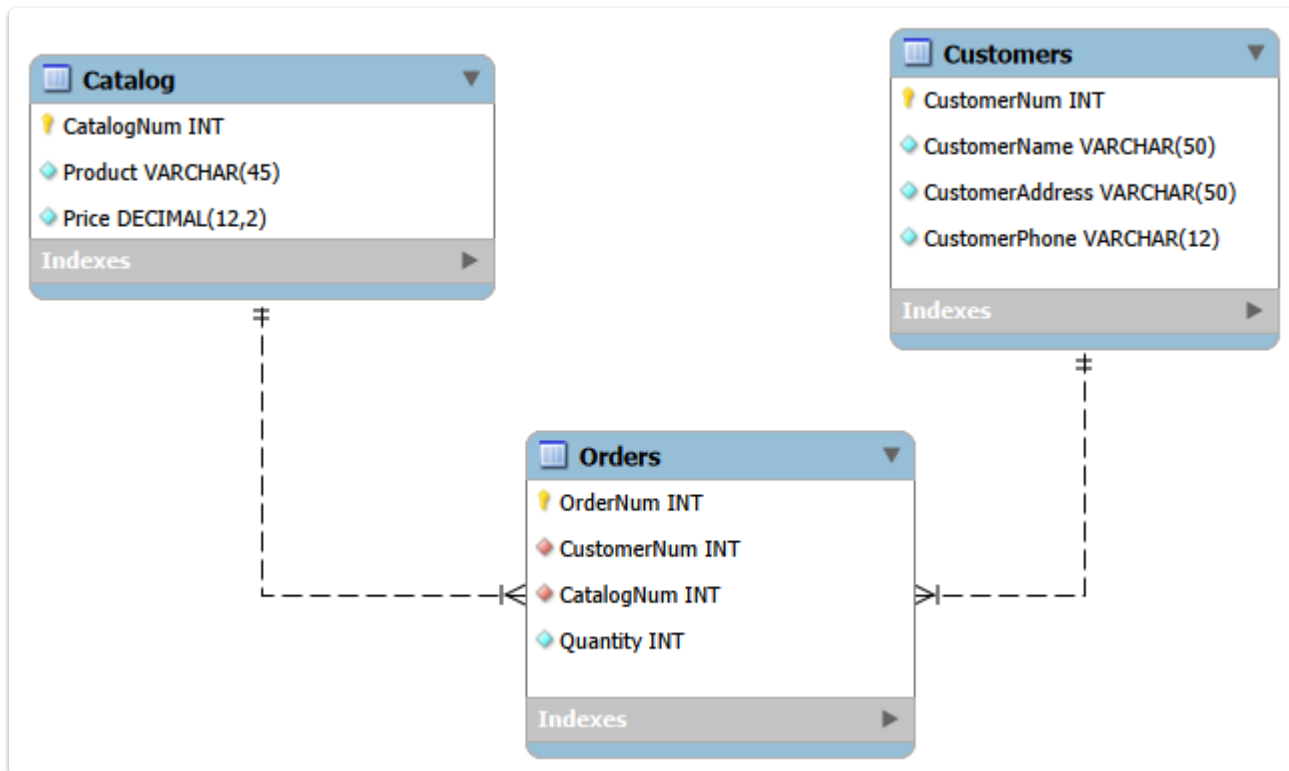


Самостоятельная работа 4. Фролов А.А.

Задание 4.1



В ходе выполнения лабораторной работы была разработана реляционная база данных, предназначенная для хранения информации о заказчиках, товарах из каталога и оформленных заказах. Исходная группированная таблица была приведена к набору отношений, соответствующих требованиям третьей нормальной формы.

При проектировании были выделены три сущности: **Customers**, **Catalog** и **Orders**.

Таблица **Customers** предназначена для хранения информации о заказчиках. Каждый заказчик идентифицируется уникальным номером `CustomerNum`. Также в таблице хранятся фамилия заказчика, его адрес и номер телефона. Так как у одного заказчика может быть только один адрес и один телефон, эти данные зависят только от первичного ключа таблицы и не дублируются в заказах.

Таблица **Catalog** предназначена для хранения информации о товарах. Каждый товар идентифицируется уникальным каталожным номером `CatalogNum`. В таблице указывается название товара и его цена. Поскольку цена товара определяется только его номером в каталоге и является постоянной, она вынесена в отдельную таблицу каталога.

Таблица **Orders** предназначена для хранения информации о заказах. В ней фиксируется номер заказа, номер заказчика, номер товара из каталога и количество заказанного товара. Если в одном заказе содержится несколько

разных товаров, то для каждого товара создаётся отдельная запись в таблице **Orders**. Если в одном заказе содержится несколько единиц одного товара, то количество указывается в поле `Quantity`, а отдельные повторяющиеся записи не создаются.

Состав таблиц

Таблица Customers

Атрибут	Тип данных	Описание
CustomerNum	INT	Уникальный номер заказчика
CustomerName	VARCHAR(50)	Фамилия заказчика
CustomerAddress	VARCHAR(50)	Адрес заказчика
CustomerPhone	VARCHAR(12)	Телефон заказчика

Первичный ключ таблицы: `CustomerNum`.

Обязательными для заполнения являются поля `CustomerNum`, `CustomerName`, `CustomerAddress`, `CustomerPhone`, так как каждый заказчик должен иметь уникальный номер, фамилию, адрес и телефон.

Таблица Catalog

Атрибут	Тип данных	Описание
CatalogNum	INT	Уникальный номер товара в каталоге
Product	VARCHAR(45)	Наименование товара
Price	DECIMAL(12,2)	Цена товара

Первичный ключ таблицы: `CatalogNum`.

Обязательными для заполнения являются поля `CatalogNum`, `Product`, `Price`, так как каждый товар должен иметь уникальный номер, название и цену.

Таблица Orders

Атрибут	Тип данных	Описание
OrderNum	INT	Уникальный номер заказа
CustomerNum	INT	Номер заказчика, оформившего заказ
CatalogNum	INT	Номер товара из каталога
Quantity	INT	Количество товара в заказе

Первичный ключ таблицы: `OrderNum` .

Обязательными для заполнения являются поля `OrderNum` , `CustomerNum` , `CatalogNum` , `Quantity` , так как каждая запись должна содержать номер заказа, ссылку на заказчика, ссылку на товар и количество заказанного товара.

Первичные ключи

В разработанной базе данных были определены следующие первичные ключи:

Таблица	Первичный ключ
<code>Customers</code>	<code>CustomerNum</code>
<code>Catalog</code>	<code>CatalogNum</code>
<code>Orders</code>	<code>OrderNum</code>

Поле `CustomerNum` однозначно идентифицирует заказчика. Это необходимо, так как среди заказчиков могут встречаться однофамильцы, поэтому фамилия не может использоваться в качестве первичного ключа.

Поле `CatalogNum` однозначно идентифицирует товар. Наименование товара также не подходит для первичного ключа, так как разные товары могут иметь похожие или одинаковые названия.

Поле `OrderNum` используется для уникальной идентификации заказа.

Связи между таблицами

Между таблицами были установлены следующие связи:

Родительская таблица	Дочерняя таблица	Связь
<code>Customers</code>	<code>Orders</code>	Один заказчик может иметь несколько заказов
<code>Catalog</code>	<code>Orders</code>	Один товар может встречаться в нескольких заказах

Связь между таблицами `Customers` и `Orders` реализуется через поле `CustomerNum` . Один заказчик может оформить несколько заказов, но каждый заказ относится только к одному конкретному заказчику.

Связь между таблицами `Catalog` и `Orders` реализуется через поле `CatalogNum` . Один товар из каталога может быть включён в разные заказы, но каждая запись в таблице заказов относится к одному конкретному товару.

Внешние ключи

В таблице **Orders** были установлены внешние ключи:

Поле	Ссылается на
CustomerNum	Customers(CustomerNum)
CatalogNum	Catalog(CatalogNum)

Поле **CustomerNum** в таблице **Orders** является внешним ключом и ссылается на первичный ключ **CustomerNum** таблицы **Customers**.

Поле **CatalogNum** в таблице **Orders** является внешним ключом и ссылается на первичный ключ **CatalogNum** таблицы **Catalog**.

Соответствие третьей нормальной форме

Разработанная база данных соответствует требованиям третьей нормальной формы.

Во-первых, все атрибуты таблиц являются атомарными, то есть в каждой ячейке хранится только одно значение. Например, адрес, телефон, товар, цена и количество представлены отдельными полями.

Во-вторых, все неключевые атрибуты зависят от первичных ключей своих таблиц. В таблице **Customers** фамилия, адрес и телефон зависят от номера заказчика. В таблице **Catalog** наименование товара и цена зависят от каталожного номера товара. В таблице **Orders** номер заказчика, номер товара и количество относятся к конкретному заказу.

В-третьих, в таблицах отсутствуют транзитивные зависимости. Цена товара не хранится в таблице заказов, так как она зависит от номера товара, а не от номера заказа. Данные о заказчике также не дублируются в таблице заказов, так как адрес и телефон зависят только от заказчика.

Задание 4.2. Простые запросы SQL

Создание таблиц через Forward Engineer:

```
1  -- MySQL Workbench Forward Engineering
2
3  SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0;
4  SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0;
5  SET @OLD_SQL_MODE=@@SQL_MODE,
    SQL_MODE='ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_
6
```

```

7  -- -----
8  -- Schema mydb
9  -- -----
10
11 -- -----
12 -- Schema mydb
13 -- -----
14 CREATE SCHEMA IF NOT EXISTS `shop` DEFAULT CHARACTER SET utf8 ;
15 USE `shop` ;
16
17 -- -----
18 -- Table `mydb`.`Catalog`
19 -- -----
20 CREATE TABLE IF NOT EXISTS `shop`.`Catalog` (
21     `CatalogNum` INT NOT NULL AUTO_INCREMENT,
22     `Product` VARCHAR(45) NOT NULL,
23     `Price` DECIMAL(12,2) NOT NULL,
24     PRIMARY KEY (`CatalogNum`),
25     UNIQUE INDEX `Product_UNIQUE` (`Product` ASC) VISIBLE)
26 ENGINE = InnoDB;
27
28
29 -- -----
30 -- Table `mydb`.`Customers`
31 -- -----
32 CREATE TABLE IF NOT EXISTS `shop`.`Customers` (
33     `CustomerNum` INT NOT NULL AUTO_INCREMENT,
34     `CustomerName` VARCHAR(50) NOT NULL,
35     `CustomerAddress` VARCHAR(50) NOT NULL,
36     `CustomerPhone` VARCHAR(12) NOT NULL,
37     PRIMARY KEY (`CustomerNum`),
38     UNIQUE INDEX `CustomerAddress_UNIQUE` (`CustomerAddress` ASC) VISIBLE,
39     UNIQUE INDEX `CustomerPhone_UNIQUE` (`CustomerPhone` ASC) VISIBLE)
40 ENGINE = InnoDB;
41
42
43 -- -----
44 -- Table `mydb`.`Orders`
45 -- -----
46 CREATE TABLE IF NOT EXISTS `shop`.`Orders` (
47     `OrderNum` INT NOT NULL AUTO_INCREMENT,
48     `CustomerNum` INT NOT NULL,
49     `CatalogNum` INT NOT NULL,
50     `Quantity` INT NOT NULL,
51     PRIMARY KEY (`OrderNum`),
52     INDEX `CustomerNum_idx` (`CustomerNum` ASC) VISIBLE,
53     INDEX `CatalogNum_idx` (`CatalogNum` ASC) VISIBLE,
54     CONSTRAINT `CustomerNum`
55     FOREIGN KEY (`CustomerNum`)

```

```

56     REFERENCES `shop`.`Customers` (`CustomerNum`)
57     ON DELETE NO ACTION
58     ON UPDATE NO ACTION,
59     CONSTRAINT `CatalogNum`
60     FOREIGN KEY (`CatalogNum`)
61     REFERENCES `shop`.`Catalog` (`CatalogNum`)
62     ON DELETE NO ACTION
63     ON UPDATE NO ACTION)
64     ENGINE = InnoDB;
65
66
67     SET SQL_MODE=@OLD_SQL_MODE;
68     SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS;
69     SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS;

```

Добавляем товары:

```

1  insert into catalog (product, price)
2  values ("PlayStation 5", 59990),
3         ("Iphone 17 Pro Max", 129990),
4         ("Dyson Pink Edition", 39990);
5
6  select * from catalog;

```

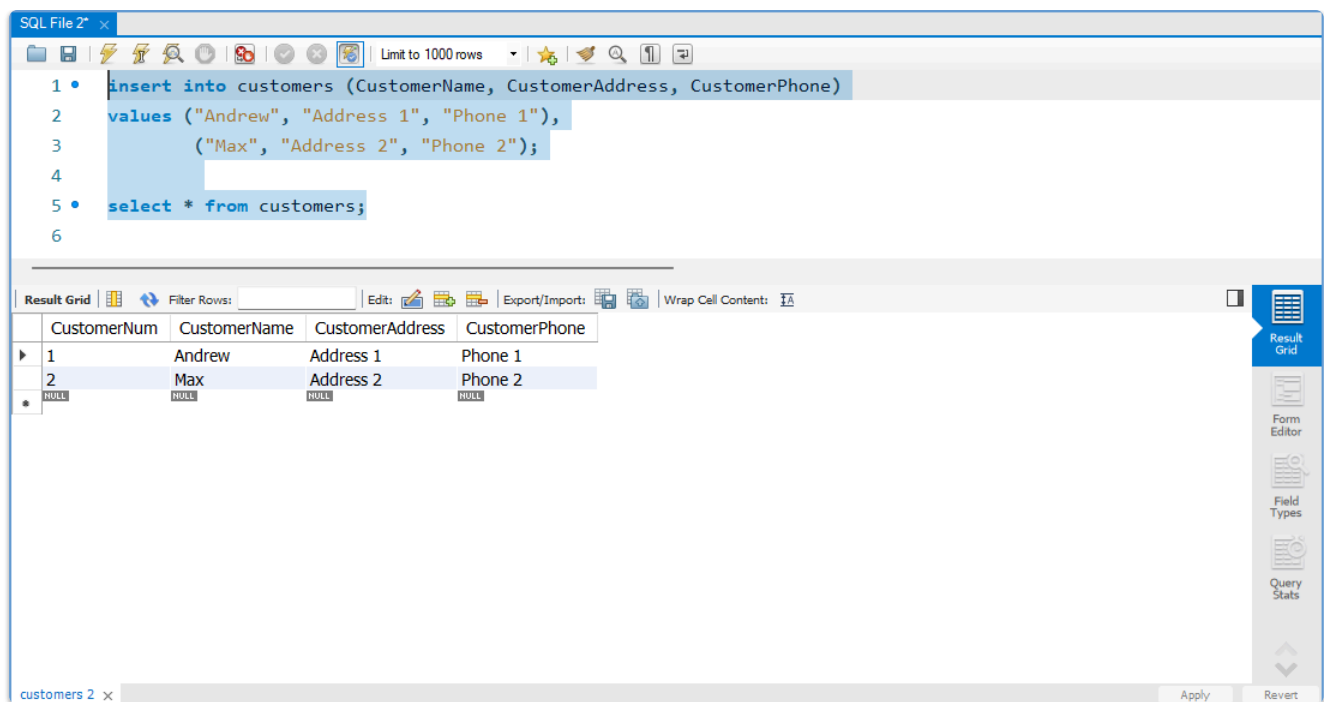
	CatalogNum	Product	Price
▶	1	PlayStation 5	59990.00
	2	Iphone 17 Pro Max	129990.00
	3	Dyson Pink Edition	39990.00
*	NULL	NULL	NULL

Добавляем клиентов:

```

1  insert into customers (CustomerName, CustomerAddress, CustomerPhone)
2  values ("Andrew", "Address 1", "Phone 1"),
3         ("Max", "Address 2", "Phone 2");
4
5  select * from customers;

```

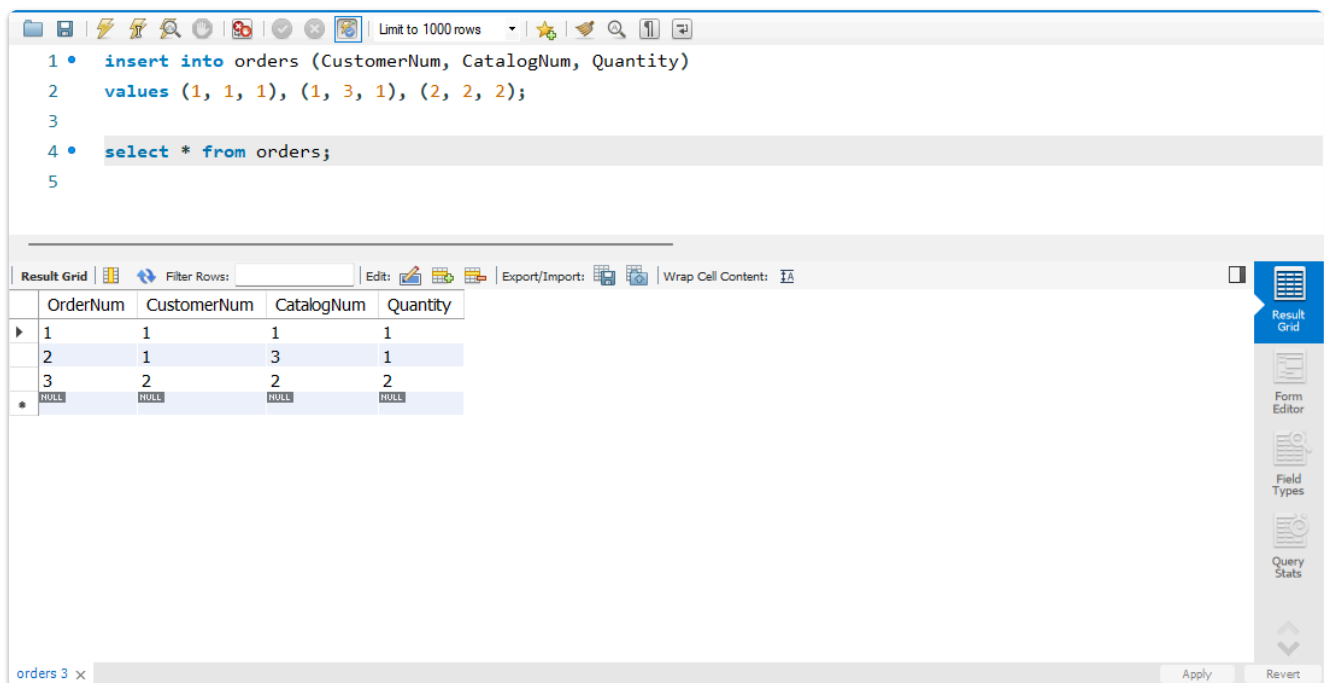


Добавляем заказ:

```

1  insert into orders (CustomerNum, CatalogNum, Quantity)
2  values (1, 1, 1), (1, 3, 1), (2, 2, 2);
3
4  select * from orders;

```



Выведем данные по заказам:

```

1  select OrderNum, CustomerName, Product, Quantity, Quantity *
2     catalog.Price as Price from
3  orders
4  inner join catalog using(CatalogNum)
5  inner join customers using(CustomerNum);

```

SQL File 2*

Limit to 1000 rows

```

1 • select OrderNum, CustomerName, Product, Quantity, Quantity * catalog.Price as Price from
2 orders
3 inner join catalog using(CatalogNum)
4 inner join customers using(CustomerNum);
5

```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	OrderNum	CustomerName	Product	Quantity	Price
▶	1	Andrew	PlayStation 5	1	59990.00
	2	Andrew	Dyson Pink Edition	1	39990.00
	3	Max	Iphone 17 Pro Max	2	259980.00

Result Grid

Form Editor

Field Types

Query Stats

Result 4

Read Only